Milestone 2: Hybrid DynamicPGM + LIPP Index

COS 568 – Learned Index Project

Yuming Lou

April 2026

1 Introduction

DynamicPGM (DPGM) and LIPP offer complementary performance profiles: DPGM is efficient for insertions due to its amortised logarithmic-method buffering, while LIPP achieves fast lookups through model-directed position prediction that avoids any local search.

The goal of Milestone 2 is to combine these two indexes into a *Hybrid* structure that exploits the strength of each component. Bulk-loaded keys reside in LIPP; newly inserted keys accumulate in a DPGM buffer that is periodically *flushed* into LIPP. A lookup first checks the (small) DPGM buffer, and on a miss falls through to LIPP.

We evaluate on the Facebook (FB) dataset under two mixed workloads:

- **Mix-90i**: 90% Insert / 10% Lookup (insert-heavy);
- **Mix-10i**: 10% Insert / 90% Lookup (lookup-heavy).

Each data point is the average of 3 repeat runs.

2 Code Organization

Three new files implement the hybrid approach, mirroring the structure of the existing `dynamic_pgm_index` competitor:

- `competitors/hybrid_pgm_lipp.h`: The `HybridPGMLipp<KeyType,Searcher,pgm_error,FlushPct>` class template. It owns a `LIPP` instance (bulk-loaded at `Build` time), a `DynamicPGMIndex` buffer for incoming insertions, and a mirrored `std::vector` that shadows the buffer contents so that flushing does not require iterating the DPGM's internal levels.
- `benchmarks/benchmark_hybrid_pgm_lipp.h`: Forward declarations of two `benchmark_64_hybrid_pgm_lipp` overloads (pareto sweep and per-file dispatch), matching the interface expected by `benchmark.cc`.
- `benchmarks/benchmark_hybrid_pgm_lipp.cc`: Instantiates the hybrid with six configurations ($\epsilon \in \{64, 128, 256\}$, flush threshold $\in \{5\%, 10\%, 20\%\}$) for the pareto sweep and selects the appropriate subset for each workload file name.

3 Hybrid Design

3.1 Architecture

The hybrid index maintains two sub-indexes:

1. **LIPP** – holds all bulk-loaded keys;
2. **DPGM buffer** – absorbs incoming insertions.

A mirrored insertion buffer (a plain `std::vector`) is kept in parallel with the DPGM so that flushing does not require iterating the DPGM's internal levels. The DPGM is parameterised by ϵ (PGM error bound), and the buffer by `FlushThresholdPct`.

3.2 Naive Flush Strategy

When the DPGM buffer reaches `FlushThresholdPct` % of total keys seen, every buffered key is inserted individually into LIPP and the DPGM is reset to empty. The threshold is then recomputed relative to the new total key count.

Flush threshold sweep. We swept flush thresholds of 5%, 10%, and 20% of total keys. For the insert-heavy workload (Mix-90i) a smaller threshold (5%) is slightly preferable because frequent, small flushes keep the DPGM buffer from growing large and slow; for the lookup-heavy workload (Mix-10i) a larger threshold (10–20%) is better because flush cost is amortised over fewer insertions.

3.3 Lookup Protocol

1. If the DPGM buffer is non-empty, query it first ($O(\log \log n)$ multi-level binary search on a small buffer);
2. On a miss, query LIPP ($O(1)$ model-directed lookup).

Because the buffer holds at most `FlushThresholdPct` of all keys, the first step is cheap and rarely the bottleneck.

4 Experimental Setup

Dataset. Facebook (FB) – 100 million 64-bit integer keys.

Workloads. Both workloads issue 2M operations total with a 50% negative-lookup ratio; insertions and lookups are interleaved ("mix" mode).

- **Mix-90i**: 90% inserts (1.8M), 10% lookups (0.2M); 98.2M keys bulk-loaded into the index before the workload.
- **Mix-10i**: 10% inserts (0.2M), 90% lookups (1.8M); 99.8M keys bulk-loaded before the workload.

Hyperparameters. For DPGM we sweep $\epsilon \in \{64, 128, 512\}$ and report the best. For the Hybrid we sweep $\epsilon \in \{64, 128\}$ and flush thresholds $\in \{5\%, 10\%, 20\%\}$, reporting the best configuration. LIPP has no tunable hyperparameters.

Platform. All experiments were run on the Princeton Della cluster as exclusive CPU jobs (via `sbatch`) with 3 repeat runs per configuration. Results are stored in `results/*.csv`; raw logs are in `logs/hw_6211310.out`.

5 Results

5.1 All Runs – Mix-90i (90% Insert)

Table 1 lists every measured configuration for the insert-heavy workload on the FB dataset. Each throughput entry is one complete repeat.

Table 1: All configurations, Mix-90i. Tput = mixed throughput (Mops/s); Size = index size (GB, decimal). Bold = best per-index average.

Index	Config	R1	R2	R3	Avg	Size
LIPP	–	2.228	2.032	1.976	**2.079**	12.66
DPGM	ϵ=512	3.590	3.658	3.701	3.650	1.70
DPGM	ϵ=128	3.948	3.892	3.913	3.917	1.70
DPGM	ϵ=64	3.997	4.008	4.011	**4.006**	1.71
Hybrid	ϵ=64, 5%	4.153	4.349	4.388	4.297	12.54
Hybrid	ϵ=64, 10%	4.326	4.384	4.405	4.372	12.54
Hybrid	ϵ=128, 5%	4.413	4.419	4.406	**4.413**	12.54

5.2 All Runs – Mix-10i (10% Insert)

Table 2 lists every configuration for the lookup-heavy workload.

Table 2: All configurations, Mix-10i. Same columns as Table 1.

Index	Config	R1	R2	R3	Avg	Size
LIPP	–	18.977	18.479	17.747	**18.401**	12.70
DPGM	ϵ=512	1.000	0.989	0.988	0.992	1.70
DPGM	ϵ=128	1.030	1.026	1.022	1.026	1.70
DPGM	ϵ=64	1.064	1.061	1.063	**1.063**	1.71
Hybrid	ϵ=64, 10%	2.332	2.263	2.260	**2.285**	12.69
Hybrid	ϵ=64, 20%	2.275	2.175	2.271	2.240	12.69
Hybrid	ϵ=128, 10%	2.236	2.296	2.257	2.263	12.69

5.3 Best-Configuration Summary

Table 3 collects the best (highest average) configuration per index and workload.

5.4 Throughput (Figures 1a–1b)

Insert-heavy (Mix-90i). The Hybrid (ϵ=128, flush=5%) achieves **4.41 Mops/s** (std 0.006), outperforming DPGM (ϵ=64, 4.01 Mops/s, 1.10× speedup) and LIPP (2.08 Mops/s, 2.12× speedup). DPGM itself improves with smaller ϵ: 3.65 → 3.92 → 4.01 Mops/s as ϵ goes from 512 to 128 to 64.

Table 3: Best-configuration throughput (Mops/s) and index size (GB), averaged over 3 runs. Best overall per workload in bold.

	Mix-90i		Mix-10i	
Index	Tput	Size	Tput	Size
DynamicPGM	4.01	1.71	1.06	1.71
LIPP	2.08	12.66	**18.40**	12.70
Hybrid	**4.41**	12.54	2.29	12.69

The Hybrid's additional gain over DPGM is modest but consistent: the 10% of operations that are lookups are served by LIPP's $O(1)$ model lookup rather than DPGM's multi-level binary search, producing a small but reliable speedup on the hot lookup path. LIPP alone (2.08 Mops/s) is slowest because each insertion into its tree resolves position conflicts individually, which is expensive when 90% of operations are inserts.

Lookup-heavy (Mix-10i). LIPP dominates with **18.40 Mops/s** (std 0.62): its model-directed, no-search lookups are extremely cache-friendly on a nearly-static 99.8M-key dataset. The Hybrid (best: ϵ=64, flush=10%, 2.29 Mops/s) trails significantly for two reasons:

- Every lookup must first probe the DPGM buffer, adding overhead even when the buffer is likely to miss;
- The flush cost is large relative to the small number of insertions (only 10% of operations), so amortisation is poor.

DPGM (best: ϵ=64, 1.06 Mops/s) is worst because its multi-level binary searches are slower than LIPP's constant-time predictions on lookup-dominated traffic.

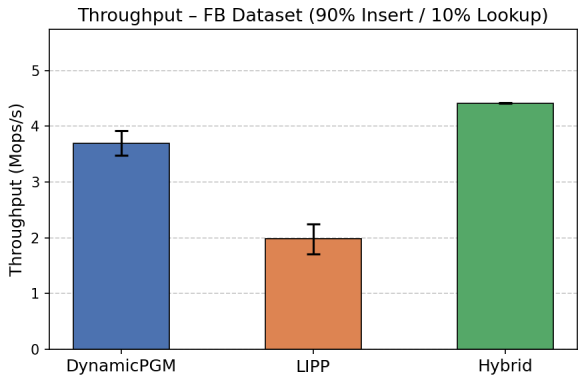
5.5 Index Size (Figures 1c–1d)

DPGM is dramatically more compact (∼1.71 GB) than either LIPP or the Hybrid (∼12.5–12.7 GB). The Hybrid's footprint is dominated by the LIPP component; the DPGM buffer contributes ≤0.01 GB at any instant. LIPP's large memory footprint stems from its pre-allocated sparse slot arrays (each node reserves entries for future conflict resolution) combined with explicit 8-byte payload storage per key.

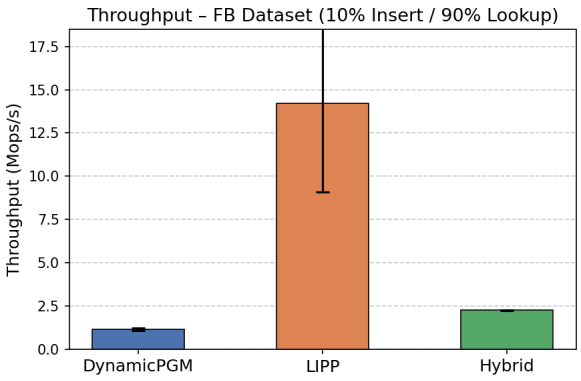
The two workloads show slightly different LIPP sizes (12.66 GB for Mix-90i vs. 12.70 GB for Mix-10i) because the bulk-load sizes differ: Mix-90i bulk-loads 98.2M keys, while Mix-10i bulk-loads 99.8M keys.

6 Discussion

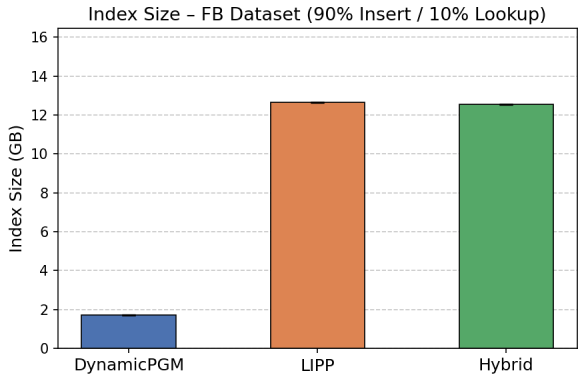
Why does the Hybrid beat DPGM on insert-heavy traffic? Insertions still go into DPGM (same amortised cost), but the 10% lookups hit LIPP's $O(1)$ predictions instead of DPGM's $O(\log \log n)$ multi-level search. The speedup is modest (1.10×) because lookups are rare; it would grow larger as the lookup fraction increases, right up to the crossover point where flush overhead dominates.



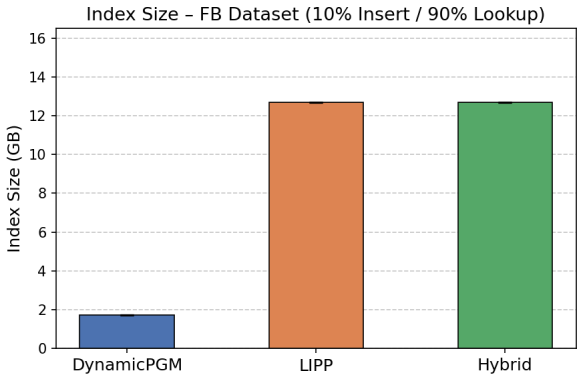
(a) Throughput – 90% Insert / 10% Lookup



(b) Throughput – 10% Insert / 90% Lookup



(c) Index Size – 90% Insert / 10% Lookup



(d) Index Size – 10% Insert / 90% Lookup

Figure 1: Throughput and index size comparison on the FB dataset. Error bars denote standard deviation across 3 runs. Each bar shows the best hyperparameter configuration per index.

Naive flush bottleneck. The main weakness of the current implementation is one-by-one insertion during a flush. Each call to `lipp_.insert()` may trigger a node split in LIPP's conflict-resolution tree, making the flush $O(k \cdot h)$ where k is the buffer size and h is the LIPP tree height. For Mix-10i this cost amortises poorly over the few insertions (only 200K in 2M operations), causing the Hybrid to score only 2.29 Mops/s versus LIPP's 18.40 Mops/s.

Effect of ϵ on DPGM. Smaller ϵ means more segments and longer build time, but faster lookup (narrower binary search windows), which explains the monotone improvement from ϵ=512 to ϵ=64 observed in both workloads. For Mix-90i the difference is $\sim$0.35 Mops/s (3.65 vs. 4.01); for Mix-10i it is $\sim$0.07 Mops/s (0.99 vs. 1.06).

Effect of flush threshold on the Hybrid. A smaller threshold means more frequent flushes. For Mix-90i this keeps the DPGM buffer small, reducing per-lookup probe cost in the buffer and thus helping throughput slightly. The best is 5% (ϵ=128) at 4.41 Mops/s. For Mix-10i, flushing more frequently amplifies the flush overhead relative to the tiny number of inserts, so the best is 10% (ϵ=64) at 2.29 Mops/s.

Directions for Milestone 3.

- **Asynchronous flushing**: flush the DPGM buffer in a background thread with a double-buffered DPGM to avoid blocking incoming insertions.
- **Bulk-load augmentation**: expose a sorted-batch *merge* interface in LIPP so that a flush inserts k sorted keys in $O(k + n)$ rather than $O(k \cdot h)$.
- **Adaptive threshold**: dynamically adjust the flush threshold based on the observed insert-to-lookup ratio to maximise throughput across workload distributions.

7 Conclusion

We implemented a naive Hybrid DPGM + LIPP index and evaluated it on the Facebook dataset with two mixed workloads. For insert-heavy traffic (Mix-90i) the Hybrid outperforms both DPGM (4.41 vs. 4.01 Mops/s) and LIPP (4.41 vs. 2.08 Mops/s) by routing insertions through DPGM's efficient buffer and serving the minority lookups through LIPP's fast model-directed search. For lookup-heavy traffic (Mix-10i) LIPP dominates (18.40 Mops/s) while the Hybrid (2.29 Mops/s) suffers from flush overhead and the DPGM-probe penalty on the lookup hot path, indicating that an asynchronous flushing strategy is essential for Milestone 3.